

GALXY · MODULE 5 PLAYBOOK

Tailored Git Workflow & Parallel Build Execution Plan

Project Scope: AI Preview Generation

Target Stack: Flask / Gemini API / Cloudinary (backend) · React (frontend)

Team Structure: 2 Backend / 2 Frontend Split

Document Version: v2.0 (Strict Execution — Updated Ownership)

This execution guide translates the global 5-Day Delivery Workflow and Git Monorepo rules into an operational step-by-step framework for the Module 5 Engineering Team, updated for the current 2-backend / 2-frontend ownership split. Every team member must follow this calendar, naming architecture, and code contract precisely to eliminate Day 4 integration failures.

UPDATED (v2.0): Ownership changed from the original 4-way T1/T2/T3/T4 backend split. Naveen now owns T1+T2 end-to-end, Gokul now owns T3+T4 end-to-end, and Sawthi and Mohammed have moved to the two frontend components (AIPreviewPanel and AIGenerationHistory) rather than backend tasks T2 and T4. Branch names and sync commands below reflect this.

1. Team Branch Assignments & Task Taxonomy

To preserve parallel development speed without mutual blocks, work is distributed across four owner-tracks: two combined backend branches and two frontend branches. All branches fork from the unified module-5-ai-preview parent branch.

Team Member	Role	Assigned Git Feature Branch(es)	Core Scope & Implementation Rules
NAVEEN KUMAR D	Backend Member 1 (Core Engine + Prompt/Provider)	feat-m5-t1-core-prompt (covers T1 + T2 work; merge feat-m5-t2-validation-cache into this branch, or keep as a sub-branch you own end-to-end)	Orchestration engine (ai_service.py), rate-limit/cache dispatch order, prompt assembly from category templates (prompt_builder_service.py), Gemini API client wrapper (ai_provider_client.py), and injection threat sanitization on custom_text.
GOKUL B	Backend Member 2 (Caching + Rate Limiting/API)	feat-m5-t3-rate-limiting (covers T3 + T4 work; merge feat-m5-t4-routes-cloudinary into this branch, or keep as a sub-branch you own end-to-end)	Deterministic configuration hashing + global cache logic (ai_cache_service.py), custom_text bypass guards, two-tier rate limiting (ai_rate_limit_service.py) with guest session_id UUID tracking and daily authenticated caps, plus ai_routes.py history routes / admin_ai_routes.py.
SAWTHI M	Frontend Member 1 (AIPreviewPanel)	feat-m5-frontend1-aipreviewpanel	Product-page AIPreviewPanel: Product Photos / AI Preview tab toggle, loading/success/rate-limit states, Regenerate button, and the stale-preview banner. Consumes Backend Member 1's POST /api/ai/generate-preview and Backend Member 2's limit_scope contract.

Team Member	Role	Assigned Git Feature Branch(es)	Core Scope & Implementation Rules
MOHAMMED	Frontend Member 2 (AIGenerationHistory)	feat-m5-frontend2-aigenerationhistory	Horizontal-scroll AIGenerationHistory strip: thumbnail render, click-to-restore configuration state, and pagination scaffolding. Consumes Backend Member 2's GET /api/ai/generations/:user_id.

2. Micro-Timeline & Daily Execution Cadence

Day 1 — Base Initialization & Shared Assets (Isolation Build)

- AM Kickoff: Initialize local environments. Pull the unified dev branch from origin and establish your feature branch locally (Naveen → feat-m5-t1-core-prompt, Gokul → feat-m5-t3-rate-limiting, Sawthi → feat-m5-frontend1-aipreviewpanel, Mohammed → feat-m5-frontend2-aigenerationhistory).
- Shared Utility Ingestion: Immediately pull down Module 1's auth decorators and Module 10's Cloudinary base helper (backend) or shared upload widget (frontend) as soon as they are pushed to dev. Do not write localized duplicates.
- Seed Ingestion: Naveen and Gokul execute python seed_categories.py inside the backend folder to provide a persistent database fixture for testing custom properties. Sawthi and Mohammed pull the same fixture's mock API responses for local component development.

Day 2 — Feature Maturity & First Synchronization Loop

- AM Synchronization (Mandatory): Sync with the project-wide development stream by running the exact merge checklist below before writing code.
- Parallel Coding: Naveen builds T1 orchestration + T2 prompt/provider together since T1 calls T2 directly. Gokul builds T3 caching + T4 rate-limiting together since both are guard layers checked in the same order. Sawthi and Mohammed build against Naveen/Gokul's documented interface contracts (see the four role handoff docs), using mocked responses until the real endpoints land.
- Focus heavily on individual test coverage using mock interfaces on all four branches.

Day 3 — Internal Integration & Feature Freeze

- AM Synchronization: Perform the final scheduled merge from origin/dev to catch external schema drifts early.
- PM Self-Integration Check: Merge all four branches into the parent module-5-ai-preview local branch. Test full end-to-end telemetry (Route → Rate Limit → Cache Check → Prompt Gen → Gemini Mock → Cloudinary Asset Pipeline → AIPreviewPanel render → AIGenerationHistory render).

Watch out: ai_routes.py is touched by both Naveen (POST /api/ai/generate-preview) and Gokul (GET history/admin routes). Agree on merge order in advance, or split into ai_routes.py + ai_history_routes.py registered under the same router, to avoid a Day 3 merge conflict.

- 21:00 Deadline: Freeze features. Push finalized parent branch to origin and flag readiness to the Project Head.

Day 4 — Global Integration Gate

- Standby Protocol: All Module 5 developers must remain active to support the Project Head during sequential integration mapping. Module 5 merges 6th in order. Fix semantic bugs instantly if escalated.

3. Mandatory Command Reference & Commit Rules

Commit messages must follow the exact syntax specification to allow automated chronological sorting:

```
[module-5] type: short description
```

Day 1 — Initialize Task Branch Off the Parent Module Branch

```
# Naveen (Backend Member 1 – T1 + T2)
git checkout -b feat-m5-t1-core-prompt origin/module-5-ai-preview

# Gokul (Backend Member 2 – T3 + T4)
git checkout -b feat-m5-t3-rate-limiting origin/module-5-ai-preview

# Sawthi (Frontend Member 1 – AIPreviewPanel)
git checkout -b feat-m5-frontend1-aipreviewpanel origin/module-5-ai-preview

# Mohammed (Frontend Member 2 – AIGenerationHistory)
git checkout -b feat-m5-frontend2-aigenerationhistory origin/module-5-ai-preview
```

Day 2 & Day 3 AM — Mandatory Sync Process (run on your own branch)

```
git checkout module-5-ai-preview
git fetch origin
git merge origin/dev
git checkout <your-feature-branch>
git merge module-5-ai-preview
```

Acceptable Conventional Commit Format Standard

```
git commit -m "[module-5] feat: implement dynamic placeholder parser for category prompts"
git commit -m "[module-5] fix: enforce strict sanitization on custom_text inputs"
git commit -m "[module-5] feat: add limit_scope-aware rate-limit banner to AIPreviewPanel"
git commit -m "[module-5] style: apply dark neon design system toast triggers"
```

4. Critical Integration Guardrails

Guardrail: Module 5 does NOT author category attribute schemas — owned by Module 2. The affects `_ai_preview` flag and all attribute labels are read-only inputs for both backend prompt-building (Naveen) and frontend tab logic (Sawthi).

Guardrail: Module 5 does NOT validate configurations manually — validation delegates entirely to Module 4's `validate_attributes()`. Naveen's pipeline calls it first, before rate-limit or cache checks.

Guardrail: Module 5 does NOT calculate financial pricing metrics — pricing estimates are pulled independently via Module 4.

Guardrail: Do NOT build `PUT /api/admin/categories/:id/ai-prompt-template` — that route belongs to Module 2, even though it is adjacent to Gokul's `admin_ai_routes.py` work.

Watch out: Frontend contracts are now first-class handoff artifacts: Sawthi depends on Naveen's response shape (`output_image_url`, `from_cache`, `generation_id`) and Gokul's 429 shape (`limit_reached`, `limit_scope`). Mohammed depends on Gokul's `GET /api/ai/generations/:user_id` shape. Confirm these field names in writing before Day 2 coding begins, since a mismatch here is now a cross-track (not just cross-branch) integration failure.